BÁO CÁO THỰC HÀNH

Thực hành An toàn mạng máy tính

Lab 01: MẬT MÃ HỌC CỔ ĐIỂN

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 29/09/2026

Nhóm 7 – NT101.R12.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
00000000	Trương Đức Thịnh	Câu	50%
00000001	Phạm Thanh Thiên	Câu 3, câu 5	50%

Link GitHub: https://github.com/truongducthinhuit-cloud/NT101.R12.1-ThucHanh

Nội dung thực hành

Nhiệm vụ 2.1: Một chương trình giải mã và mã hóa bằng Ceasar cipher

- Ngôn ngữ lựa chọn: C++
- Môi trường : VS studio 2022

CODE:

```
#include <iostream>
#include <string>

using namespace std;

//Ham ma hoa Caesar
string encryptCaesar(string text, int key) {
    string result = "";
    key = key % 26;
    if (key < 0) key += 26;

    for (int i = 0; i < text.length(); i++) {
        char c = text[i];
```

```
        if (isupper(c)) {
            result += char(int(c + key - 'A') % 26 + 'A');
        }
        else if (islower(c)) {
            result += char(int(c + key - 'a') % 26 + 'a');
        }
        else {
            result += c;
        }
    }
    return result;
}
//Ham giai ma Caesar
string decryptCaesar(string text, int key) {
    return encryptCaesar(text, 26 - (key % 26));
}

//Ham Brute-force tim ban duy nhat
void bruteForceCaesar(string ciphertext) {
    int bestKey = 0;
    int maxScore = -1;
    string bestPlaintext = "";

    for (int k = 1; k < 26; k++) {
        string decrypted = decryptCaesar(ciphertext, k);

        int score = 0;
        if (decrypted.find(" the ") != string::npos || decrypted.find("The ") != string::npos)
score++;
        if (decrypted.find(" and ") != string::npos) score++;
        if (decrypted.find(" is ") != string::npos || decrypted.find(" are ") != string::npos)
score++;
        if (decrypted.find(" in ") != string::npos) score++;
        if (decrypted.find(" a ") != string::npos) score++;

        if (score > maxScore) {
            maxScore = score;
            bestKey = k;
            bestPlaintext = decrypted;
        }
    }

    cout << "\nDa tim thay ban chinh xac nhat:" << endl;
    cout << "-> Khoa k = " << bestKey << endl;
```

```
    cout << "-> Van ban:\n" << bestPlaintext << endl;
}

int main() {
    int choice;
    cout << "1. Ma hoa / Giai ma thu cong" << endl;
    cout << "2. Brute-force" << endl;
    cout << "Nhap lua chon(1 hoac 2): ";
    cin >> choice;
    cin.ignore();

    if (choice == 1) {
        string text;
        int key;
        int mode;

        cout << "\nNhap van ban: ";
        getline(cin, text);

        cout << "Nhap khoa k (so nguyen): ";
        cin >> key;

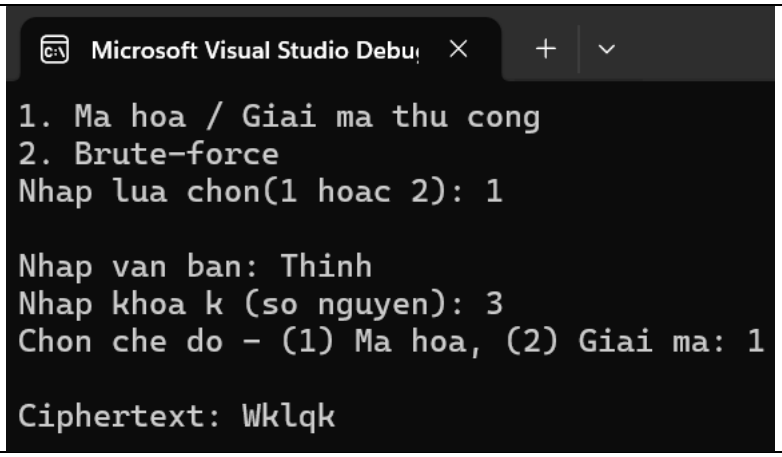
        cout << "Chon che do - (1) Ma hoa, (2) Giai ma: ";
        cin >> mode;

        if (mode == 1) {
            cout << "\nCiphertext: " << encryptCaesar(text, key) << endl;
        }
        else if (mode == 2) {
            cout << "\nPlaintext: " << decryptCaesar(text, key) << endl;
        }
        else {
            cout << "Khong hop le!" << endl;
        }
    }
//brute
    else if (choice == 2) {
        string ciphertext;
        cout << "\nNhap doan ciphertext (luu y copy-paste tren 1 dong lien mach): \n";
        getline(cin, ciphertext);
        bruteForceCaesar(ciphertext);
    }
    else {
        cout << "Lua chon khong hop le!" << endl;
```

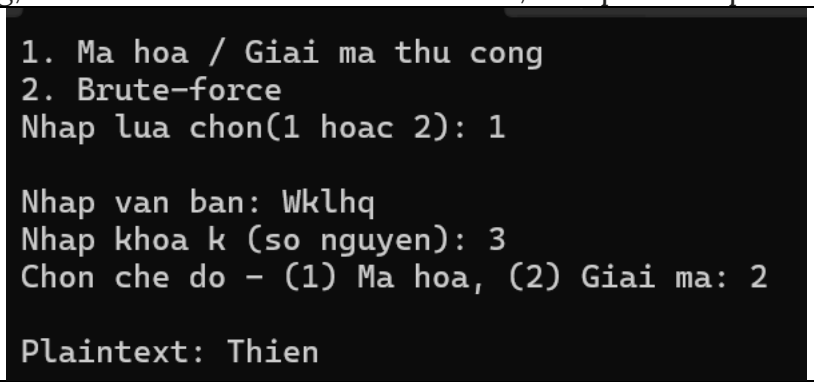
```
    }

    return 0;
}
```

- Test case yêu cầu 1



Mã hóa thủ công, khóa K = 3 mã hóa văn bản Thinh, kết quả: Wklqk



Giải mã thủ công, khóa K = 3 giải mã văn bản Wklhq, kết quả: Thien

- Testcase yêu cầu 2

```
1. Ma hoa / Giai ma thu cong
2. Brute-force
Nhap lua chon(1 hoac 2): 2

Nhap doan ciphertext (luu y copy-paste tren 1 dong lien mach):
Max NBM bl t extwbgz bglmbmnmbhg ngwxk OGN-AVF, lixvbtebsbgz bg max ybxew hy bgyhkftmbhg mxvaghehzr. Xlmtueblaxw pbma t
fbllbhg mh yhlmxk bgghotmbhg tgw xqvxeexgvx bg BM xwnvtmbhg tgw kxlxtkva, NBM hyyxkl t pbwx ktgzx hy ngwxkzktwntmx tgw i
hlmzktwntmx ikhzktfl tbfxw tm ikhwnvbgz abzaer ldbeexw ikhyxllbhgtel. Max ngboxklbmr bl kxvhzgbsxw yhk bml vnmmbgz-xwzx
kxlxtkva bg tkxtl ebdx vruxklxvnkbmr, tkmbybvbte bgmxeebzxgvx, tgw lhymptkx xgzbgxxkbgz. Pbma lmtmx-hy-max-tkm ytvbebmbx
l tgw t lmkhgz xfiatlbl hg vheetuhktmbhg pbma bgwnlmkr, NBM xjnbil lmnwxgml pbma uhma maxhkxmbvte dghpexwzx tgw iktvmbvt
e ldbeel mh makbox bg max ktibwer xoheobgz mxva bgwnlmkr.

Da tim thay ban chinh xac nhat:
-> Khoa k = 19
-> Van ban:
The UIT is a leading institution under VNU-HCM, specializing in the field of information technology. Established with a
mission to foster innovation and excellence in IT education and research, UIT offers a wide range of undergraduate and p
ostgraduate programs aimed at producing highly skilled professionals. The university is recognized for its cutting-edge
research in areas like cybersecurity, artificial intelligence, and software engineering. With state-of-the-art facilitie
s and a strong emphasis on collaboration with industry, UIT equips students with both theoretical knowledge and practica
l skills to thrive in the rapidly evolving tech industry.
```

Nhiệm vụ 2.2: Khôi phục lại bản rõ bằng phương pháp frequency analysis và mô tả chi tiết quy trình thực hiện theo từng bước

Bước 1: Chuẩn bị và thông kê tần suất (frequency counting)

- Sử dụng **Tabular N-gram Analysis**
- Lần lượt cho N-gram =1, 2, 3
- Thu được bảng sau:

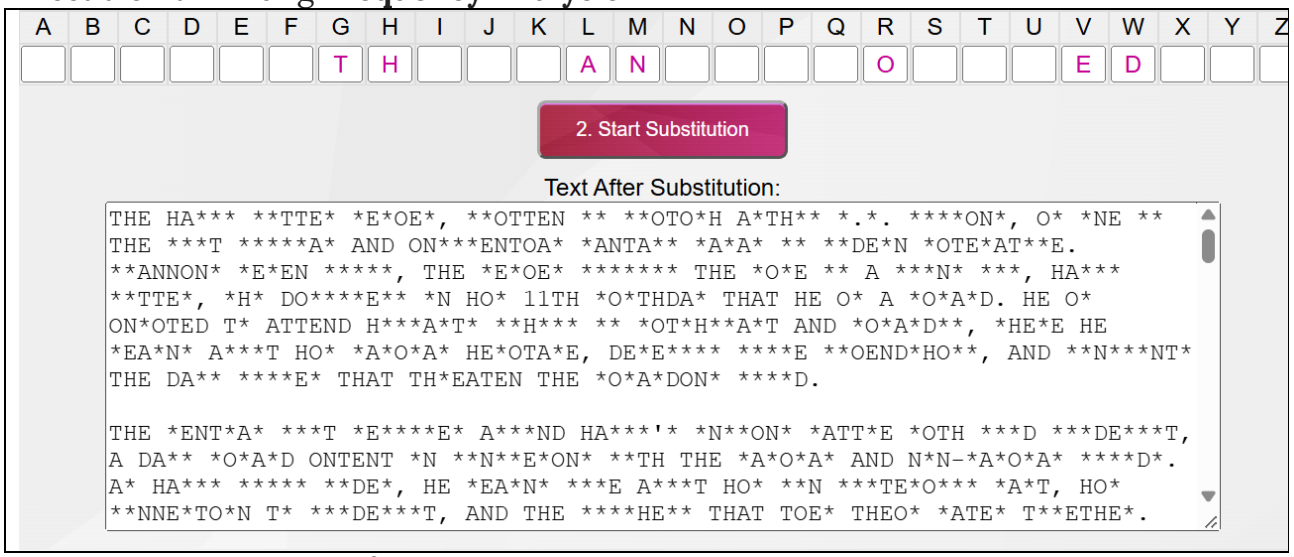
N-gram tables

Rank	1-gram	Abs.	Rel.
1	v	486	10.163
2	r	398	8.323
3	m	384	8.030

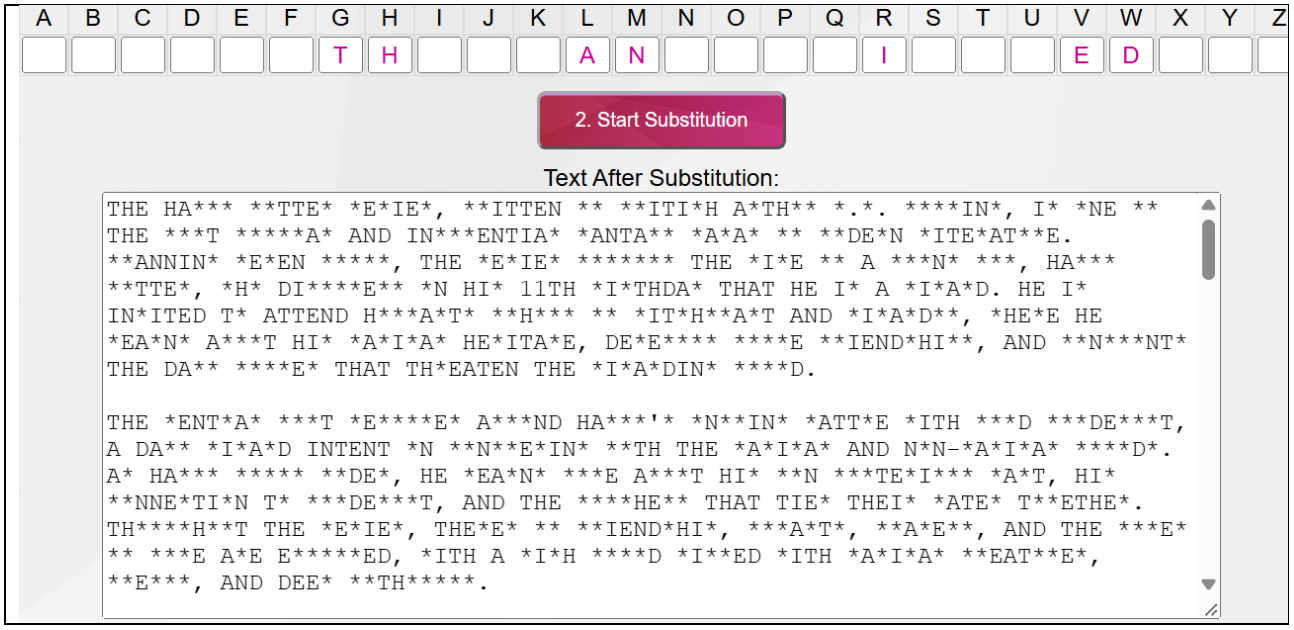
Rank	2-gram	Abs.	Rel.
1	rm	97	2.028
2	vs	88	1.840
3	vi	83	1.736

Rank	3-gram	Abs.	Rel.
1	lmw	54	1.129
2	ghv	38	0.795
3	rmt	37	0.774

- Đặt giả thuyết
    + Chữ E là chữ xuất hiện nhiều nhất trong tiếng anh nên V có thể là E
    + Cụm GHV thay E và là __E có thể là từ THE nên G = T, H = H
    + Cụm LMW có M đứng hạng 3 trong tần suất 1 ký tự có thể cụm AND, L = A, M = N,W = D
    + Cụm RM có thể là IS hoặc ON
- Test trên tính năng **Frequency Analysis**



- Trường hợp R = O thì bảng giải mã khá bí, nên ta sẽ thử trường hợp R = I
- Ví dụ HE O* ON*OTED sai ngữ pháp và vô nghĩa



- Phân tích ***IETNA* NATI*NA* *NI*E**IT*** có thể là **VIETNAM NATIONAL UNIVERSITY**. Từ đây có thể lấy thêm được các chữ **V, M, L, U**.
- **ED**ATI*N:** có thể là **EDUCATION** (mở khóa chữ **C**).
- THE HA*** **TTE* *E*IE*: Nằm ở ngay câu mở đầu, đây chắc chắn là cụm **THE HARRY POTTER SERIES**. Giải mã được ngay các chữ cái **R, Y, P, O, S**.
- **ITTEN ** **ITI*H A*TH**: Cụm này là **WRITTEN BY BRITISH AUTHOR** (giúp mở khóa chữ **W, B**).

Đoạn văn bản được giải mã là

THE HARRY POTTER SERIES, WRITTEN BY BRITISH AUTHOR J.K. ROWLING, IS ONE OF THE MOST POPULAR AND INFLUENTIAL FANTASY SAGAS IN MODERN LITERATURE. SPANNING SEVEN NOVELS, THE SERIES FOLLOWS THE LIFE OF A YOUNG BOY, HARRY POTTER, WHO DISCOVERS ON HIS 11TH BIRTHDAY THAT HE IS A WIZARD. HE IS INVITED TO ATTEND HOGWARTS SCHOOL OF WITCHCRAFT AND WIZARDRY, WHERE HE LEARNS ABOUT HIS MAGICAL HERITAGE, DEVELOPS CLOSE FRIENDSHIPS, AND CONFRONTS THE DARK FORCES THAT THREATEN THE WIZARDING WORLD.

THE CENTRAL PLOT REVOLVES AROUND HARRY'S ONGOING BATTLE WITH LORD VOLDEMORT, A DARK WIZARD INTENT ON CONQUERING BOTH THE MAGICAL AND NON-MAGICAL WORLDS. AS HARRY GROWS OLDER, HE LEARNS MORE ABOUT HIS OWN MYSTERIOUS PAST, HIS CONNECTION TO VOLDEMORT, AND THE PROPHECY THAT TIES THEIR FATES TOGETHER. THROUGHOUT THE SERIES, THEMES OF FRIENDSHIP, LOYALTY, BRAVERY, AND THE POWER OF LOVE ARE EXPLORED, WITH A RICH WORLD FILLED WITH MAGICAL CREATURES, SPELLS, AND DEEP MYTHOLOGY.

THE NOVELS, STARTING WITH HARRY POTTER AND THE PHILOSOPHER'S STONE (1997) AND CONCLUDING WITH HARRY POTTER AND THE DEATHLY HALLOWS (2007), HAVE SOLD OVER 500 MILLION COPIES WORLDWIDE, BEEN TRANSLATED INTO MORE THAN 80 LANGUAGES, AND ADAPTED INTO A HIGHLY SUCCESSFUL FILM SERIES. THE SERIES IS KNOWN NOT ONLY FOR ITS ENCHANTING WORLD-

BUILDING BUT ALSO FOR ADDRESSING DARKER, COMPLEX THEMES SUCH AS MORTALITY, PREJUDICE, AND THE ABUSE OF POWER. THE HARRY POTTER SERIES HAS LEFT AN INDELIBLE MARK ON POPULAR CULTURE, FOSTERING A MASSIVE FAN BASE, INSPIRING COUNTLESS SPIN-OFFS, AND CEMENTING ITS PLACE AS A LITERARY PHENOMENON.

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY (VNU-HCM) IS ONE OF THE MOST PRESTIGIOUS AND COMPREHENSIVE HIGHER EDUCATION INSTITUTIONS IN VIETNAM, KNOWN FOR ITS EXCELLENCE IN RESEARCH, INNOVATION, AND EDUCATION. FOUNDED IN 1995 BY THE VIETNAMESE GOVERNMENT, VNU-HCM WAS ESTABLISHED AS A KEY CENTER FOR TRAINING, RESEARCH, AND TECHNOLOGY TRANSFER WITH THE MISSION OF CONTRIBUTING TO THE COUNTRY'S SOCIO-ECONOMIC DEVELOPMENT. AS A FLAGSHIP UNIVERSITY, IT HAS PLAYED A VITAL ROLE IN PRODUCING HIGHLY SKILLED PROFESSIONALS, SCIENTISTS, AND LEADERS ACROSS MULTIPLE FIELDS.

VNU-HCM OPERATES UNDER A DISTINCTIVE STRUCTURE AS A MULTI-DISCIPLINARY AND MULTI-CAMPUS SYSTEM, COMPRISING SEVERAL MEMBER UNIVERSITIES AND RESEARCH INSTITUTES. THESE INCLUDE SOME OF THE MOST WELL-ESTABLISHED INSTITUTIONS IN THE REGION, SUCH AS THE UNIVERSITY OF SCIENCE, UNIVERSITY OF TECHNOLOGY, UNIVERSITY OF SOCIAL SCIENCES AND HUMANITIES, INTERNATIONAL UNIVERSITY, UNIVERSITY OF ECONOMICS AND LAW, AND OTHERS. EACH OF THESE MEMBER INSTITUTIONS OPERATES SEMI-INDEPENDENTLY BUT COLLABORATES UNDER THE UMBRELLA OF VNU-HCM, MAKING IT A UNIQUE HIGHER EDUCATION MODEL IN VIETNAM THAT PROMOTES INTERDISCIPLINARY LEARNING AND RESEARCH.

LOCATED IN HO CHI MINH CITY, THE COUNTRY'S ECONOMIC AND CULTURAL HUB, VNU-HCM BENEFITS FROM CLOSE TIES WITH INDUSTRIES, BUSINESSES, AND GOVERNMENT AGENCIES. THIS PROXIMITY FOSTERS COLLABORATION IN RESEARCH, INTERNSHIPS, AND JOB PLACEMENT, ENSURING THAT ITS GRADUATES ARE WELL-PREPARED TO MEET THE DEMANDS OF AN EVOLVING WORKFORCE. THE UNIVERSITY IS COMMITTED TO PRODUCING GRADUATES WHO ARE NOT ONLY ACADEMICALLY PROFICIENT BUT ALSO EQUIPPED WITH PRACTICAL SKILLS AND A GLOBAL OUTLOOK.

VNU-HCM IS RENOWNED FOR ITS STRONG EMPHASIS ON SCIENTIFIC RESEARCH AND TECHNOLOGICAL INNOVATION. IT HOSTS NUMEROUS RESEARCH CENTERS, INSTITUTES, AND LABS THAT FOCUS ON VARIOUS AREAS OF SCIENCE, ENGINEERING, SOCIAL SCIENCES, ECONOMICS, AND MORE. THE UNIVERSITY'S RESEARCH AGENDA IS ALIGNED WITH NATIONAL PRIORITIES, FOCUSING ON AREAS SUCH AS INFORMATION TECHNOLOGY, BIOTECHNOLOGY, RENEWABLE ENERGY, AND ENVIRONMENTAL SUSTAINABILITY. VNU-HCM ALSO ACTIVELY ENGAGES IN INTERNATIONAL COOPERATION, PARTNERING WITH LEADING UNIVERSITIES AND RESEARCH

INSTITUTIONS WORLDWIDE TO ENHANCE ITS ACADEMIC PROGRAMS, FOSTER JOINT RESEARCH PROJECTS, AND PROMOTE STUDENT AND FACULTY EXCHANGES.
THE UNIVERSITY IS ALSO DEEPLY INVOLVED IN ADDRESSING GLOBAL CHALLENGES THROUGH ITS RESEARCH AND EDUCATIONAL PROGRAMS. VNU-HCM PLACES A STRONG EMPHASIS ON SUSTAINABLE DEVELOPMENT, ENVIRONMENTAL CONSERVATION, AND TECHNOLOGICAL INNOVATION, ALIGNING ITS GOALS WITH THE UNITED NATIONS' SUSTAINABLE DEVELOPMENT GOALS (SDGS). THE INSTITUTION ENCOURAGES ITS FACULTY AND STUDENTS TO PURSUE RESEARCH THAT HAS A MEANINGFUL IMPACT ON BOTH THE LOCAL AND GLOBAL COMMUNITY.
IN TERMS OF STUDENT LIFE, VNU-HCM OFFERS A VIBRANT AND DYNAMIC ENVIRONMENT. WITH A STUDENT BODY OF OVER 60,000 AND A FACULTY OF HIGHLY QUALIFIED PROFESSORS AND RESEARCHERS, THE UNIVERSITY PROVIDES A RICH ACADEMIC AND SOCIAL EXPERIENCE. THE CAMPUSES ARE EQUIPPED WITH MODERN FACILITIES, INCLUDING LIBRARIES, LABORATORIES, SPORTS COMPLEXES, AND DORMITORIES, CREATING A SUPPORTIVE LEARNING ENVIRONMENT. STUDENTS ALSO HAVE ACCESS TO A VARIETY OF EXTRACURRICULAR ACTIVITIES, CLUBS, AND ORGANIZATIONS, ALLOWING THEM TO ENGAGE IN CULTURAL, SOCIAL, AND PROFESSIONAL DEVELOPMENT ALONGSIDE THEIR STUDIES.
IN CONCLUSION, VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY STANDS AS A LEADING EDUCATIONAL INSTITUTION IN VIETNAM, CONTRIBUTING SIGNIFICANTLY TO THE COUNTRY'S INTELLECTUAL, SCIENTIFIC, AND ECONOMIC PROGRESS. ITS COMMITMENT TO ACADEMIC EXCELLENCE, RESEARCH INNOVATION, AND SOCIETAL IMPACT HAS POSITIONED IT AS A CRUCIAL PLAYER IN SHAPING THE FUTURE OF VIETNAM AND THE BROADER REGION. AS IT CONTINUES TO EVOLVE AND EXPAND, VNU-HCM REMAINS DEDICATED TO FOSTERING THE NEXT GENERATION OF LEADERS AND INNOVATORS WHO WILL DRIVE SUSTAINABLE GROWTH AND GLOBAL PROGRESS.

Nhiệm vụ 2.3: xây dựng một chương trình hỗ trợ giải mã tự động mono-alphabetic substitution cipher khi chỉ biết ciphertext.

Phương pháp thực hiện:

- Xây dựng chương trình C++ tự động phá mã bằng thuật toán **Hill-Climbing kết hợp Random Restart**.
- Sử dụng hàm đánh giá (Fitness Function) dựa trên tần suất ký tự (Unigram) và các cụm từ phổ biến (Bigram, Trigram) để đo lường độ giống tiếng Anh của văn bản.

Kết quả kiểm thử và Hạn chế gặp phải

Hiện tượng: Chương trình hoàn tất quá trình tìm kiếm nhưng văn bản trả về vẫn chứa nhiều ký tự vô nghĩa hoặc chỉ dò đúng một vài từ đơn lẻ (như "THE"), chưa thể khôi phục hoàn toàn 100% bản rõ gốc.

Nguyên nhân:

+ Mắc kẹt ở cực đại cục bộ (Local Optimum): Khi thuật toán tình cờ dò trúng một vài cụm từ phổ biến, điểm Fitness tăng cao khiến chương trình bị "khóa" các ký tự đó và mất khả năng tối ưu phần còn lại.
+ Hạn chế của hàm Fitness: Mô hình chấm điểm đơn giản chưa đủ độ nhạy ngữ nghĩa, dễ bị đánh lừa bởi các chuỗi ký tự có tổng điểm "ảo".
+ Độ dài dữ liệu: Các đoạn ciphertext ngắn thiếu dữ liệu thống kê tần suất, khiến thuật toán dễ đi lệch hướng.

CODE:

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <random>
#include <cmath>

using namespace std;

// Chuẩn hóa dữ liệu: Chỉ giữ lại chữ cái in hoa
string normalize_text(const string& text) {
    string norm = "";
    for (char c : text) {
        if (isalpha(c)) norm += toupper(c);
    }
    return norm;
}

// HÀM FITNESS NÂNG CẤP 2 LỚP: Tần suất đơn (Unigram) + Cụm từ (N-gram)
double calculate_fitness(const string& norm_text) {
    double score = 0;
    double len = norm_text.length();
    if (len == 0) return 0;

    // 1. Đếm tần suất xuất hiện của từng chữ cái trong đoạn văn bản hiện tại
    int counts[26] = { 0 };
    for (char c : norm_text) counts[c - 'A']++;

    // Tần suất chuẩn của 26 chữ cái tiếng Anh (tính theo %)
    double expected_freq[26] = { 8.167, 1.492, 2.782, 4.253, 12.702, 2.228, 2.015, 6.094,
6.966, 0.153, 0.772, 4.025, 2.406, 6.749, 7.507, 1.929, 0.095, 5.987, 6.327, 9.056, 2.758,
0.978, 2.360, 0.150, 1.974, 0.074 };

    // Tính tổng sai số (Penalty). Sai số càng thấp, văn bản càng giống tiếng Anh.
```

```
    double penalty = 0;
    for (int i = 0; i < 26; i++) {
        double actual = (counts[i] / len) * 100.0;
        penalty += abs(actual - expected_freq[i]);
    }
    score -= penalty * 3.0; // Trừ điểm nếu sai số phân bố cao

    // 2. Chấm điểm cộng dựa trên các cụm từ phổ biến (N-grams)
    const vector<string> TRIGRAMS = {
"THE","AND","ING","ENT","ION","HER","FOR","THA","NTH","INT","ERE","TIO","TER","E
ST","ERS","ATI","HAT","ATE","ALL","ETH","PRO" };
    const vector<string> BIGRAMS = {
"TH","HE","IN","ER","AN","RE","ON","AT","EN","ND","TI","ES","OR","TE","OF","ED","IS","I
T","AL","AR","ST","TO","NT","NG","SE","HA","AS" };

    for (const string& tg : TRIGRAMS) {
        size_t pos = 0;
        while ((pos = norm_text.find(tg, pos)) != string::npos) { score += 4.0; pos++; }
    }
    for (const string& bg : BIGRAMS) {
        size_t pos = 0;
        while ((pos = norm_text.find(bg, pos)) != string::npos) { score += 1.0; pos++; }
    }

    return score;
}

// Giải mã chuỗi dựa trên khóa
string decrypt(const string& cipher, const string& key) {
    string plain = "";
    for (char c : cipher) {
        if (isalpha(c)) plain += key[toupper(c) - 'A'];
        else plain += c;
    }
    return plain;
}

void solve_monoalphabetic(const string& original_ciphertext) {
    string norm_cipher = normalize_text(original_ciphertext);
    string best_global_key = "";
    double best_global_score = -999999;

    random_device rd;
    mt19937 gen(rd());
```

```
    int NUM_RESTARTS = 50;
    for (int restart = 0; restart < NUM_RESTARTS; restart++) {
        string current_key = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        shuffle(current_key.begin(), current_key.end(), gen);

        string current_plain = decrypt(norm_cipher, current_key);
        double current_score = calculate_fitness(current_plain);

        int no_improve = 0;
        while (no_improve < 2500) {
            string new_key = current_key;
            swap(new_key[gen() % 26], new_key[gen() % 26]);


            if (gen() % 100 < 10) {
                swap(new_key[gen() % 26], new_key[gen() % 26]);
            }

            string new_plain = decrypt(norm_cipher, new_key);
            double new_score = calculate_fitness(new_plain);

            if (new_score > current_score) {
                current_score = new_score;
                current_key = new_key;
                no_improve = 0;
            }
            else {
                no_improve++;
            }
        }

        if (current_score > best_global_score) {
            best_global_score = current_score;
            best_global_key = current_key;
        }
    }

    cout << "\n================ KET QUA ================" << endl;
    cout << "=> Key tim duoc: " << best_global_key << endl;
    cout << "=> Ban ro (Plaintext):\n" << decrypt(original_ciphertext,
best_global_key) << "\n";
}
```

```
int main() {
    string ciphertext, line;
    cout << "Nhap ciphertext (Nhan Enter 2 lan de bat dau giai ma):\n";

    while (getline(cin, line) && !line.empty()) {
        ciphertext += line + "\n";
    }

    if (ciphertext.empty()) return 0;

    cout << "\nDang ap dung Hill-Climbing Nhanh (1-3 giay), vui long doi..." << endl;
    solve_monoalphabetic(ciphertext);

    return 0;
}
```

Testcase : PEHCGBTENCUL VF GUR CENPGVPR NAQ FGHQL BS GRPUAVDHRF SBE FRPHER PBZZHAVPNGVBA VA GUR CERFRAPR BS NQIREFNEVNY ORUNIVBE. ZBER TRARENYYL, PEHCGBTENCUL VF NOBHG PBAFGEHPGVAT NAQ NANYZVAT CEBGBPBYF GUNG CERIRAG GUVEQ CNEGVRF BE GUR CHOYVP SEBZ ERNQVAT CEVINGR ZRFFNTRF.

```
Nhap ciphertext (Nhan Enter 2 lan de bat dau giai ma):
PEHCGBTENCUL VF GUR CENPGVPR NAQ FGHQL BS GRPUAVDHRF SBE FRPHER PBZZHAVPNGVBA VA GUR CERFRAPR BS NQIREFNEVNY ORUNIVBE. Z
BER TRARENYYL, PEHCGBTENCUL VF NOBHG PBAFGEHPGVAT NAQ NANYZVAT CEBGBPBYF GUNG CERIRAG GUVEQ CNEGVRF BE GUR CHOYVP SEBZ E
RNQVAT CEVINGR ZRFFNTRF.


Dang ap dung Hill-Climbing Nhanh (1-3 giay), vui long doi...

================ KET QUA ================
=> Key tim duoc: NOLKRSTCWZQYJABDUEPGHIXVMF
=> Ban ro (Plaintext):
DRCLTOGRALHY IS THE LRADTIDE ANU STCUY OP TEDHNIKCES POR SEDCRE DOFFCNIDATION IN THE LRESENDE OP AUWERSARIAM BEHAWIOR. F
ORE GENERAMMY, DRCLTOGRALHY IS ABOCT DONSTRCDTING ANU ANAMFING LROTODOMS THAT LREWENT THIRU LARTIES OR THE LCBMID PROF R
EAUING LRIWATE FESSAGES.


E:\code\ANTOANMANG\Auto_mono-alphabetic_substitution_cipher\x64\Debug\Auto_mono-alphabetic_substitution_cipher.exe (proc
ess 19252) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Đánh giá: Có thể xuất ra các từ như IS, THE, THAT, IN nhưng vẫn không ra được bản rõ hoa

Nhiệm vụ 2.4: **Viết một chương trình để mã hóa và giải mã bằng Playfair cipher.**

CODE:

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <cctype>

using namespace std;

// Chuẩn hóa chuỗi (chỉ lấy chữ cái in hoa, thay J bằng I)
string prepare_text(const string& text) {
    string res = "";
    for (char c : text) {
        if (isalpha(c)) {
            char u = toupper(c);
            if (u == 'J') u = 'I';
            res += u;
        }
    }
    return res;
}

// Tạo ma trận 5x5 từ khóa
vector<vector<char>> generate_playfair_matrix(string key) {
    key = prepare_text(key);
    string key_processed = "";
    bool visited[26] = { false };
    visited['J' - 'A'] = true; // J được gộp với I

    for (char c : key) {
        if (!visited[c - 'A']) {
            visited[c - 'A'] = true;
            key_processed += c;
        }
    }

    for (char c = 'A'; c <= 'Z'; c++) {
        if (!visited[c - 'A']) {
            visited[c - 'A'] = true;
            key_processed += c;
        }
    }
```

```
    vector<vector<char>> matrix(5, vector<char>(5));
    int idx = 0;
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            matrix[i][j] = key_processed[idx++];
        }
    }
    return matrix;
}

// Tìm vị trí của ký tự trong ma trận
void find_position(const vector<vector<char>>& matrix, char c, int& row, int& col) {
    if (c == 'J') c = 'I';
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            if (matrix[i][j] == c) {
                row = i;
                col = j;
                return;
            }
        }
    }
}

// Chuẩn hóa bản rõ thành các cặp ký tự (digraphs)
string prepare_plaintext(string text) {
    text = prepare_text(text);
    string res = "";
    for (size_t i = 0; i < text.length(); i++) {
        res += text[i];
        if (i + 1 < text.length()) {
            if (text[i] == text[i + 1]) {
                res += 'X'; // Chèn 'X' nếu 2 ký tự trùng nhau trong một cặp
            }
            else {
                res += text[i + 1];
                i++;
            }
        }
    }
    if (res.length() % 2 != 0) {
        res += 'X'; // Thêm 'X' nếu số lượng ký tự lẻ
    }
    return res;
```

```
}

// Mã hóa Playfair
string playfair_encrypt(string plaintext, const vector<vector<char>>& matrix) {
    plaintext = prepare_plaintext(plaintext);
    string ciphertext = "";

    for (size_t i = 0; i < plaintext.length(); i += 2) {
        char c1 = plaintext[i];
        char c2 = plaintext[i + 1];
        int r1, c1_idx, r2, c2_idx;

        find_position(matrix, c1, r1, c1_idx);
        find_position(matrix, c2, r2, c2_idx);

        if (r1 == r2) { // Cùng hàng
            ciphertext += matrix[r1][(c1_idx + 1) % 5];
            ciphertext += matrix[r2][(c2_idx + 1) % 5];
        }
        else if (c1_idx == c2_idx) { // Cùng cột
            ciphertext += matrix[(r1 + 1) % 5][c1_idx];
            ciphertext += matrix[(r2 + 1) % 5][c2_idx];
        }
        else { // Hình chữ nhật
            ciphertext += matrix[r1][c2_idx];
            ciphertext += matrix[r2][c1_idx];
        }
    }
    return ciphertext;
}

// Giải mã Playfair
string playfair_decrypt(string ciphertext, const vector<vector<char>>& matrix) {
    ciphertext = prepare_text(ciphertext);
    string plaintext = "";

    for (size_t i = 0; i < ciphertext.length(); i += 2) {
        char c1 = ciphertext[i];
        char c2 = ciphertext[i + 1];
        int r1, c1_idx, r2, c2_idx;

        find_position(matrix, c1, r1, c1_idx);
        find_position(matrix, c2, r2, c2_idx);
```

```
        if (r1 == r2) { // Cùng hàng
            plaintext += matrix[r1][(c1_idx - 1 + 5) % 5];
            plaintext += matrix[r2][(c2_idx - 1 + 5) % 5];
        }
        else if (c1_idx == c2_idx) { // Cùng cột
            plaintext += matrix[(r1 - 1 + 5) % 5][c1_idx];
            plaintext += matrix[(r2 - 1 + 5) % 5][c2_idx];
        }
        else { // Hình chữ nhật
            plaintext += matrix[r1][c2_idx];
            plaintext += matrix[r2][c1_idx];
        }
    }
    return plaintext;
}

int main() {
    string key, text;
    int choice;

    cout << "=== CHUONG TRINH PLAYFAIR CIPHER ===" << endl;
    cout << "Nhap khoa (Key): ";
    getline(cin, key);

    vector<vector<char>> matrix = generate_playfair_matrix(key);

    cout << "\n--- Ma tran Playfair 5x5 ---" << endl;
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    cout << "\nChon chuc nang:" << endl;
    cout << "1. Ma hoa (Encrypt)" << endl;
    cout << "2. Giai ma (Decrypt)" << endl;
    cout << "Nhap lua chon (1 hoac 2): ";
    cin >> choice;
    cin.ignore();

    cout << "Nhap van ban: ";
    getline(cin, text);
```

```
    if (choice == 1) {
        string cipher = playfair_encrypt(text, matrix);
        cout << "\n=> Ban ma (Ciphertext):\n" << cipher << endl;
    }
    else if (choice == 2) {
        string plain = playfair_decrypt(text, matrix);
        cout << "\n=> Ban ro (Plaintext):\n" << plain << endl;
    }
    else {
        cout << "Lua chon khong hop le!" << endl;
    }

    return 0;
}
```

Testcase giải mã:

```
=== CHUONG TRINH PLAYFAIR CIPHER ===
Nhap khoa (Key): Harry Potter

--- Ma tran Playfair 5x5 ---
H A R Y P
O T E B C
D F G I K
L M N Q S
U V W X Z

Chon chuc nang:
1. Ma hoa (Encrypt)
2. Giai ma (Decrypt)
Nhap lua chon (1 hoac 2): 2
Nhap van ban: ARYWYPHCBVEBYGMPNCYGCNTDNCWTMGRMFTQPLEWTMLREFBEBQEBIYGBFLPHVOAEHKDHEUNGQFEROLEWTMLOPHEQGOSBEROQDWTLCMTHBWL
NRKXRYLORYYPHCBVEBYRLGYDMKYGGWKLROANDBWGNERMNGYRLGHEWRTRLMBRHMUDGVODVTEGMCHLGWCMTFODNRRYCMZKODDUTDXGEOPOYRMFRMGUKXRYGHAB
ROVTGQMCEHPRPEOTSEGEQLARYWYPOTMGQDOEXGOAUDHGUTULTNEHFTFHPGXGVPHGURBDMEGWKLETCBOTNTFQLTAEHMTUGEOAHEVEROXGVPHGDEWTEWGQIEDL
PILERWPMOATNGQKQEAHBMVRFKBRMKLXODXFREBHMNUKXRYKLRMFLWDDNCN

=> Ban ro (Plaintext):
HARXRYPOTXTERISASERIESOFSEVENFANTASYNOVELSWRITTENBYBRITISHAUTHORIKROWLINGTHENOVELSCHRONICLETHELIVESOFAYOUNGWIZARDHARRYPO
TXTERANDHISFRIENDSHERMIONEGRANGERANDRONWEASLEYALLOFWHOMARESTUDENTSATHOGWARTSSCHOOLOFWITCHCRAFTANDWIZARDRYTHEMAINSTORYARC
CONCERNSHARXRYCONFLICTWITHLORDVOLDEMORTADARKWIZARDWHOINTENDSTOBECOMEIMMORTALOVERTHROWTHEWIZARDGOVERNINGBODYKNOWNASTHEMIN
ISTRYOFMAGICANDSUBIUGATEALLWIZARDSANDMUGGLES

E:\code\ANTOANMANG\Playfair Cipher\x64\Debug\Playfair Cipher.exe (process 17508) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Testcase mã hóa:

THE UNIVERSITY OF INFORMATION TECHNOLOGY IS AWESOME

```
=== CHUONG TRINH PLAYFAIR CIPHER ===
Nhap khoa (Key): Harry Potter

--- Ma tran Playfair 5x5 ---
H A R Y P
O T E B C
D F G I K
L M N Q S
U V W X Z

Chon chuc nang:
1. Ma hoa (Encrypt)
2. Giai ma (Decrypt)
Nhap lua chon (1 hoac 2): 1
Nhap van ban: THE UNIVERSITY OF INFORMATION TECHNOLOGY IS AWESOME

=> Ban ma (Ciphertext):
OAOWQGWTPNFBHBGKMGEHVTBFELEBOPLEUDIRKQRVCNTLBW

E:\code\ANTOANMANG\Playfair Cipher\x64\Debug\Playfair Cipher.exe (process 13236) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Nhiệm vụ 2.5:

Code:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <cctype>

using namespace std;

// Chuan hoa khoa: chi giu chu cai, doi thanh chu hoa
string normalizeKey(const string& key) {
    string k = "";
    for (char c : key)
        if (isalpha((unsigned char)c)) k += (char)toupper((unsigned char)c);
    return k;
}

// Ma hoa / giai ma Vigenere (mode = +1: ma hoa, mode = -1: giai ma)
string vigenere(const string& text, const string& key, int mode) {
    string result = "";
    int keyIdx = 0;
    int keyLen = key.length();

    for (size_t i = 0; i < text.length(); i++) {
        char c = text[i];
        if (isalpha((unsigned char)c)) {
            char base = isupper((unsigned char)c) ? 'A' : 'a';
            int shift = key[keyIdx % keyLen] - 'A';
            int x = (c - base + mode * shift + 26) % 26;
            result += (char)(x + base);
            keyIdx++;              // chi tang khi gap chu cai
        }
        else {
            result += c;           // giu nguyen dau cach, dau cau, so
        }
    }
    return result;
}

string readFile(const string& path) {
    ifstream f(path);
    if (!f) return "";
```

```
    stringstream ss;
    ss << f.rdbuf();
    return ss.str();
}

int main() {
    string key, text, path;
    int choice, source;

    cout << "=== VIGENERE CIPHER ===\n";
    cout << "Nhap khoa (Key): ";
    cin >> key;
    key = normalizeKey(key);
    if (key.empty()) {
        cout << "Khoa khong hop le (phai co it nhat 1 chu cai)!\n";
        return 1;
    }

    cout << "Chon chuc nang (1: Ma hoa, 2: Giai ma): ";
    cin >> choice;

    cout << "Nguon van ban (1: Go truc tiep, 2: Doc tu file): ";
    cin >> source;
    cin.ignore(10000, '\n');

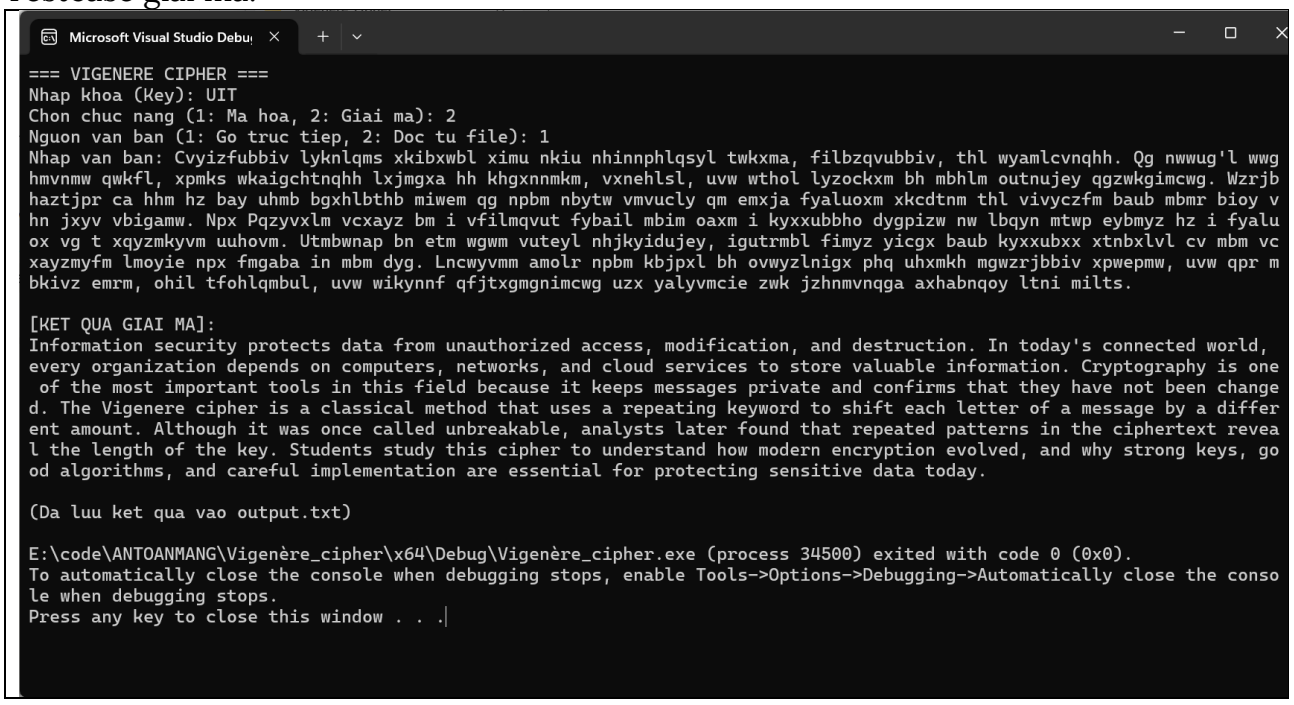
    if (source == 2) {
        cout << "Ten file (vd: input.txt): ";
        getline(cin, path);
        text = readFile(path);
        if (text.empty()) {
            cout << "Khong doc duoc file hoac file rong!\n";
            return 1;
        }
        cout << "Da doc " << text.length() << " ky tu tu file.\n";
    }
    else {
        cout << "Nhap van ban: ";
        getline(cin, text);
    }

    string out;
    if (choice == 1)      out = vigenere(text, key, +1);
    else if (choice == 2) out = vigenere(text, key, -1);
    else { cout << "Lua chon khong hop le!\n"; return 1; }
```

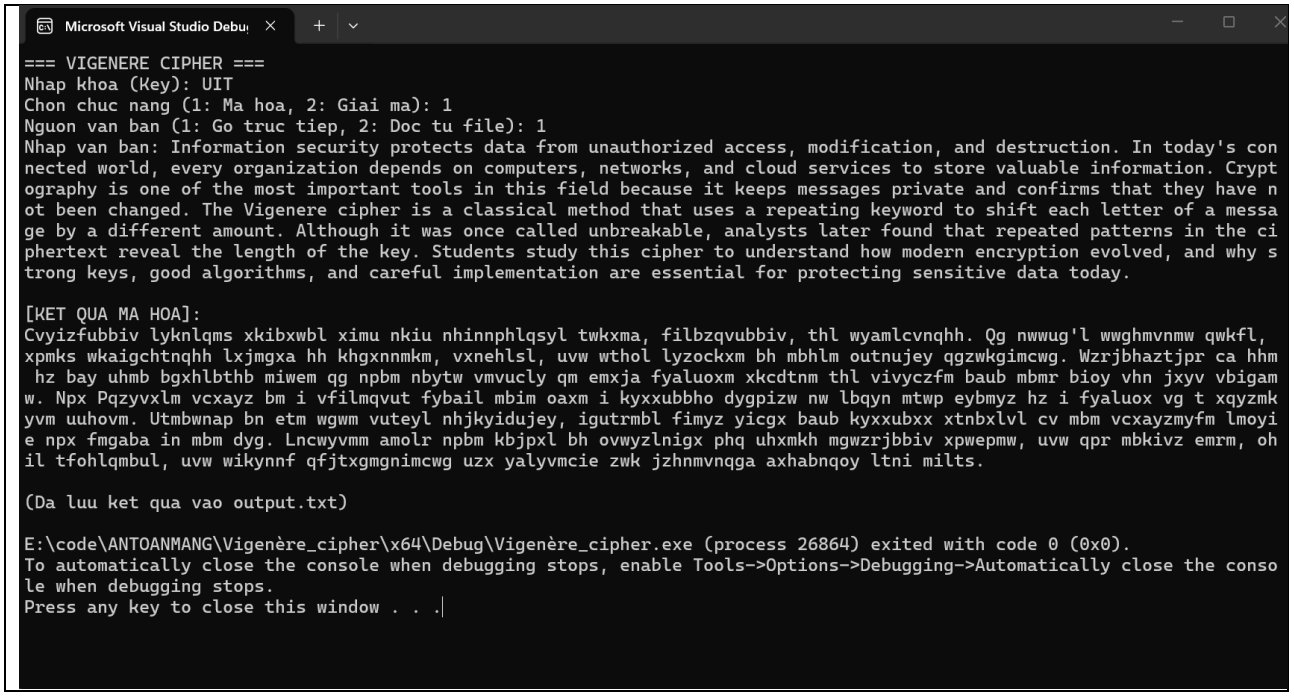
```
    cout << (choice == 1 ? "\n[KET QUA MA HOA]:\n" : "\n[KET QUA GIAI MA]:\n")
         << out << endl;

    ofstream fo("output.txt");
    fo << out;
    cout << "\n(Da luu ket qua vao output.txt)\n";
    return 0;
}
```

Testcase giải mã:



Testcase mã hóa:



Nhiệm vụ 2.6: Viết một chương trình để mã hóa và giải mã bằng Vigenere khi không biết khóa

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <cctype>
#include <cmath>
using namespace std;

// Tan suat xuat hien cua 26 chu cai trong tieng Anh (%)
const double EN[26] = { 8.167,1.492,2.782,4.253,12.702,2.228,2.015,6.094,6.966,
                        0.153,0.772,4.025,2.406,6.749,7.507,1.929,0.095,5.987,
                        6.327,9.056,2.758,0.978,2.360,0.150,1.974,0.074 };

// BUOC 1: chuan hoa - chi giu chu cai, doi thanh chu HOA
string normalize(const string& s) {
    string r;
    for (unsigned char c : s) if (isalpha(c)) r += toupper(c);
    return r;
}

// BUOC 2: Index of Coincidence cua mot chuoi
double ic(const string& s) {
    int f[26] = { 0 }, n = s.size();
    if (n < 2) return 0;
```

```
    for (char c : s) f[c - 'A']++;
    double sum = 0;
    for (int i = 0; i < 26; i++) sum += f[i] * (f[i] - 1);
    return sum / (n * (n - 1.0));
}

// IC trung binh khi tach chuoi thanh L cot
double avgIC(const string& s, int L) {
    double total = 0;
    for (int i = 0; i < L; i++) {
        string col;
        for (size_t j = i; j < s.size(); j += L) col += s[j];
        total += ic(col);
    }
    return total / L;
}

// BUOC 3: chon do dai khoa = L NHO NHAT co IC trung binh >= 0.06
// (tieng Anh ~0.065, ngau nhien ~0.038). Neu khong co thi lay L co IC lon nhat.
int guessKeyLen(const string& s, int maxL) {
    int best = 1; double bestIC = 0;
    for (int L = 1; L <= maxL; L++) {
        double v = avgIC(s, L);
        cout << "  L = " << L << "  IC = " << v << "\n";
        if (v >= 0.06) return L;
        if (v > bestIC) { bestIC = v; best = L; }
    }
    return best;
}

// BUOC 4: voi moi cot, thu 26 phep dich, chon phep dich co chi-binh-phuong nho
nhat
char guessKeyChar(const string& col) {
    int bestShift = 0; double bestChi = 1e18;
    for (int k = 0; k < 26; k++) {
        int f[26] = { 0 };
        for (char c : col) f[(c - 'A' - k + 26) % 26]++;
        double chi = 0, n = col.size();
        for (int i = 0; i < 26; i++) {
            double e = n * EN[i] / 100.0;
            chi += (f[i] - e) * (f[i] - e) / e;
        }
        if (chi < bestChi) { bestChi = chi; bestShift = k; }
    }
```

```
    return 'A' + bestShift;
}

// Giai ma, giu nguyen dau cach / dau cau cua van ban goc
string decrypt(const string& text, const string& key) {
    string r; int j = 0;
    for (unsigned char c : text) {
        if (isalpha(c)) {
            char base = isupper(c) ? 'A' : 'a';
            r += (char)((c - base - (key[j % key.size()] - 'A') + 26) % 26 + base);
            j++;
        }
        else r += c;
    }
    return r;
}

int main() {
    int mode;
    string text;
    cout << "=== PHA MA VIGENERE (khong biet khoa) ===\n";
    cout << "1: Nhap truc tiep, 2: Doc tu file: ";
    cin >> mode; cin.ignore(10000, '\n');

    if (mode == 2) {
        string path; cout << "Ten file: "; getline(cin, path);
        ifstream f(path); stringstream ss; ss << f.rdbuf(); text = ss.str();
    }
    else {
        cout << "Nhap ciphertext: "; getline(cin, text);
    }

    string c = normalize(text);
    if (c.size() < 30) { cout << "Ciphertext qua ngan!\n"; return 1; }
    cout << "So chu cai sau chuan hoa: " << c.size() << "\n";

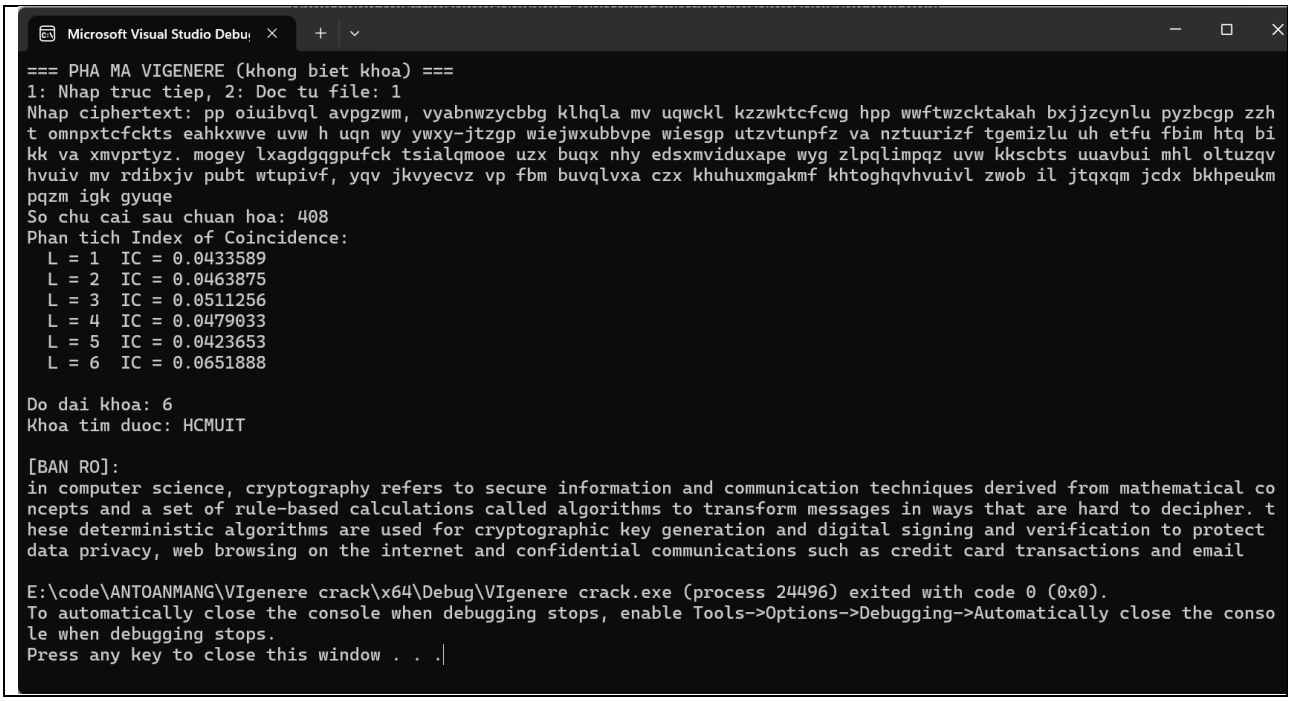
    cout << "Phan tich Index of Coincidence:\n";
    int L = guessKeyLen(c, 20);

    string key;
    for (int i = 0; i < L; i++) {
        string col;
        for (size_t j = i; j < c.size(); j += L) col += c[j];
        key += guessKeyChar(col);
```

```
    }

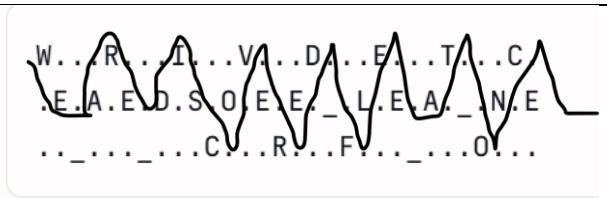
    cout << "\nDo dai khoa: " << L << "\nKhoa tim duoc: " << key << "\n";
    cout << "\n[BAN RO]:\n" << decrypt(text, key) << "\n";
    return 0;
}
```

Testcase:



Nhiệm vụ 2.7 Lựa chọn một thuật toán mật mã cổ điển bất kỳ, trình bày ngắn gọn nguyên lý hoạt động và xây dựng một chương trình để thực hiện mã hóa và giải mã theo thuật toán đã chọn.

- Loại mã hóa chọn là Rail Fence Cipher (mật mã hàng rào)
- Thuộc nhóm hoán vị (transposition): giữ nguyên các ký tự, chỉ đổi vị trí của chúng.\
- Viết bản rõ theo hình **zigzag** (đường chéo xuống rồi chéo lên) qua các hàng.
- Đọc ký tự **lần lượt từng hàng, từ trên xuống dưới**, ghép lại thành bản mã.



Code:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// Tao mau zigzag: vi tri i cua van ban nam o hang (rail) nao
// Vi du n=10, rails=3 -> 0 1 2 1 0 1 2 1 0 1
vector<int> zigzag(int n, int rails) {
    vector<int> p(n);
    int r = 0, dir = 1;
    for (int i = 0; i < n; i++) {
        p[i] = r;
        if (rails > 1) {
            if (r == 0) dir = 1;              // cham hang tren -> di xuong
            else if (r == rails - 1) dir = -1; // cham hang duoi -> di len
            r += dir;
        }
    }
    return p;
}

// MA HOA: doc lan luot tung hang, tu tren xuong duoi
string encrypt(const string& text, int rails) {
    vector<int> p = zigzag(text.size(), rails);
    string out;
    for (int r = 0; r < rails; r++)
        for (size_t i = 0; i < text.size(); i++)
            if (p[i] == r) out += text[i];
    return out;
}

// GIAI MA: biet vi tri nao thuoc hang nao -> do ky tu cua ban ma vao dung cho
string decrypt(const string& cipher, int rails) {
    vector<int> p = zigzag(cipher.size(), rails);
    string out(cipher.size(), ' ');
    int idx = 0;
    for (int r = 0; r < rails; r++)
        for (size_t i = 0; i < cipher.size(); i++)
```

```
            if (p[i] == r) out[i] = cipher[idx++];
    return out;
}

// In "hang rao" de minh hoa (dung cho bao cao)
void showFence(const string& text, int rails) {
    vector<int> p = zigzag(text.size(), rails);
    for (int r = 0; r < rails; r++) {
        for (size_t i = 0; i < text.size(); i++)
            cout << (p[i] == r ? (text[i] == ' ' ? '_' : text[i]) : '.');
        cout << "\n";
    }
}

int main() {
    int choice, rails;
    string text;
    cout << "=== RAIL FENCE CIPHER ===\n";
    cout << "So hang (rails, >= 2): ";
    cin >> rails;
    cout << "Chon (1: Ma hoa, 2: Giai ma): ";
    cin >> choice;
    cin.ignore(10000, '\n');
    if (rails < 2) { cout << "So hang phai >= 2!\n"; return 1; }

    cout << "Nhap van ban: ";
    getline(cin, text);

    if (choice == 1) {
        cout << "\n[HANG RAO]:\n";
        showFence(text, rails);
        cout << "\n[KET QUA MA HOA]:\n" << encrypt(text, rails) << "\n";
    } else {
        cout << "\n[KET QUA GIAI MA]:\n" << decrypt(text, rails) << "\n";
    }
    return 0;
}
```

Testcase mã hóa:



Testcase giải mã:

